

TriAgent

Project Dossier

A terminal orchestrator for ChatGPT + Claude + Gemini — parallel, relay, and council modes, a compact inter-agent protocol, an API-key wizard, and a plan for routing cheap work through free models.

Generated June 12, 2026

Contents

1. The Plan
2. Architecture
3. Quick Start & Usage
4. The Inter-Agent Protocol
5. free-claude-code Routing
6. Full Source

1. The Plan

One accessible multi-agent setup: use strong models where judgment matters, cheap or free models where it doesn't, skills as reusable recipes, and keep dangerous permissions away from unreliable models. Built so someone with **no budget** can still get real work done.

Three layers — they do different jobs, don't conflate them

Layer 1 — Orchestrator: triad (this repo)

- Talk to ChatGPT + Claude + Gemini at once: parallel / relay / council.
- Uses the **paid** OpenAI / Anthropic / Gemini APIs directly. **Costs money per message**.
- Use it for: comparing frontier takes, collaboration between strong models, and the council pattern for high-stakes decisions.

Layer 2 — Cheap execution: free-claude-code (separate install)

- A proxy that points **Claude Code** (a real file-editing coding agent) at free or local models, routed per tier. Repo: github.com/Alishahryar1/free-claude-code.
- **Free**, because the target user has no budget. Local is optional for those with the RAM.
- Use it for: actual coding work at \$0.
- Recommended per-tier config lives in [config/free-claude-code.env](#).

Layer 3 — Advisory sidecar: TriAgent Sidecar (separate, your build)

- Cheap models as **advisory-only** reviewers. They do **not** edit files or make final calls.
- Use it for: a cheap second opinion / code review / error summary without risking your repo.

*The big one: Layer 1 is paid, Layer 2 is free, and they don't connect today. Running triad bills your API keys; it is *not* the free-model setup. Picking the right layer for the job is the whole point.*

The decision (recorded so it doesn't get lost)

From the council, after correcting for the real constraint (a **MacBook Air**, and users who can't pay):

- **Free-cloud by default**. "Just run it locally" is the privilege answer — the target users don't have the hardware. Free cloud routing is the **point**, not the compromise.
- **Local is an opt-in** for whoever has the RAM. free-claude-code already ships Ollama / LM Studio / llama.cpp — it's one config entry, no extra code.
- **Per-tier routing is about spreading rate limits, not cost** (cost is already \$0).

Capable model on Opus/Sonnet tiers, fast model on Haiku/subagent tier, a **different** provider as fallback so one 429 doesn't kill a session.

- **Validate every model slug** in the Admin UI before trusting it — free catalogs drift.
- **Don't hand the cheapest models write + bash on anything you can't** `git reset`. Run agentic edits on a clean tree so a bad run is one command away from undone.

- The "free tiers may train on your inputs" privacy caveat only bites on **sensitive data**,

which is out of scope for this project now.

Roadmap

- [x] triad orchestrator (parallel / relay / council) + skill files
- [x] API key wizard: masked prompts, live validation, safe save, /keys, keychain option
- [x] **Free roster (Stage 1)**: run triad on free models — --free (Groq / OpenRouter / NIM,

deliberately distinct lineages) and a free Claude slot via TRIAD_CLAUDE_BASE_URL → free-claude-code (it speaks the Anthropic API, so no new agent code).

- [] **Next — the load-bearing wall: generate-verify-select**. An executable oracle (run the

tests/code) that *selects* the candidate that passes and feeds failures back to revise. Without it, free aggregation is the correlated-agreement trap; the free roster above is only substrate. The oracle bounds quality, so a critique-revise loop sits on top of selection. Honest scope: this is a force multiplier on **verifiable** tasks (code/math/structured), not subjective ones — where paid council stays the right tool.

- [] Drive free-claude-code's file-editing agent (subprocess, sandboxed on a clean git tree

so a bad run is one reset away) — a *separate* integration from the chat bridge above, and where this converges with the agent-relay-terminal work.

2. Architecture

Three independent tools that share a philosophy, not a codebase. This doc is the map so you reach for the right one and don't expect one to do another's job.

Which tool for which job

You want to...	Use	Cost	Edits files?
Compare how 3 frontier models answer the same thing	triad (parallel)	paid	no
Have models build on each other / synthesize	triad (relay/council)	paid	no
Pressure-test a real decision	triad (council)	paid	no
Get actual coding work done for free	free-claude-code	free	yes
A cheap second opinion without risking your repo	TriAgent Sidecar	cheap	no

How the pieces relate

```
+-----+
| Layer 1: triad (this repo) |
| 3 frontier models, paid APIs, chat-only |
| parallel · relay · council |
+-----+
      (no live link yet - see roadmap)
+-----+
| Layer 2: free-claude-code (separate) |
| proxy -> free/local models drive Claude Code |
| per-tier routing, edits files, runs bash |
+-----+
+-----+
| Layer 3: TriAgent Sidecar (separate) |
| cheap models = advisory only, no edits |
+-----+
```

They're decoupled on purpose. Each is useful alone; nothing breaks if you only run one.

triad internals (Layer 1)

```
cli.py          REPL + slash commands + `setup` subcommand
+-- dotenv.py    auto-load .env so keys work without `source`
+-- keychain.py  optional OS-keychain storage (graceful no-op without `keyring`)
+-- setup_keys.py masked-input wizard + live key validation
+-- agents.py    OpenAIAgent / ClaudeAgent / GeminiAgent (async streaming, lazy SDKs)
+-- orchestrator.py parallel / relay / council modes
+-- ui.py        rich Live: 3-column parallel + single-panel streaming
+-- skills.py    loads skills/*.md and applies them to agents' system prompts
```

The agent layer is the seam. To make triad itself run on free/local models later, you add one Agent subclass that points at free-claude-code's proxy (or any OpenAI-compatible endpoint) and register it in `build_agents()`. Modes, skills, UI, and the key wizard all work unchanged. That's the bridge in the roadmap.

Key handling (the part we just hardened)

Order of resolution at startup: real environment vars → .env (auto-loaded) → OS keychain (for any still missing).
python -m triad setup validates each key against the provider's /models endpoint before saving — a dead key is caught at setup, not three turns into a task.

3. Quick Start & Usage

Install

```
cd triad_project
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Configure keys

```
python -m triad setup          # masked prompts, live validation, safe save
python -m triad setup --all    # also NVIDIA NIM / Groq / OpenRouter
```

Keys auto-load on startup — no source needed. Add or fix one mid-session with `/keys`.

Run

```
python -m triad                # parallel mode
python -m triad --mode council
python -m triad --mode relay
```

Modes

- **parallel** — same task to all three at once, independent answers side by side.
- **relay** — agents work in sequence, each building on the previous output.
- **council** — all answer, then a chair agent synthesizes the best single answer.

Commands

```
/mode parallel|relay|council  switch collaboration mode
/protocol on|off              compact handoffs in relay/council (saves tokens)
/skill <name> [agent|all]     apply a skill to an agent
/skills                       list available skills
/agents                       list active agents + models
/keys                         add or fix an API key
/reset                        clear history
/save [file.md]              save transcript
/help      /quit
```

Skills

Markdown files in `skills/` with frontmatter (name, description, agents). Applying one appends its body to that agent's system prompt; stack as many as you like. Seven ship with the project: planner, architect, implementer, code-reviewer, red-team, synthesizer, concise.

4. The Inter-Agent Protocol

Relay and council normally re-send the whole growing transcript on every hop — that's the real token waste. With `/protocol` on, agents emit a terse, structured handoff and only the digest is passed forward. The full output is kept in a reference store so the human transcript loses nothing. It stays in-distribution (models already understand the format, so capability holds) and human-readable (you keep oversight).

Handoff format

```
@goal <one line: what you're solving>
@find
- <terse finding or contribution>
- <terse finding>
@conf <0.0-1.0>
@next <what's open / who should act / 'done'>
```

Why not a brand-new compressed language

A truly novel notation pushes models out-of-distribution: you get a slower, dumber agent, and you can no longer read what they're saying — bad for a human-in-the-loop design. Real machine-to-machine compression (latent vectors / "neuralese") needs models that share a representation space, which you only get with local models you run yourself — never across the OpenAI/Anthropic/Gemini text APIs. So the protocol is a compact *schema* the models already know, plus reference-passing instead of re-quoting. After a relay or council run it prints how much handoff context that saved; the saving grows with conversation length.

5. free-clause-code Routing

`free-clause-code.env` here is a **reference**, not a file `free-clause-code` reads. `fcc` manages its own settings through its Admin UI, so you apply these values there.

Apply it

- Install and start `free-clause-code`: `fcc-server`.
- Open the Admin UI URL it prints (e.g. `http://127.0.0.1:8082/admin`, `loopback-only`).
- **Providers** view: paste `NVIDIA_NIM_API_KEY`, `GROQ_API_KEY`, `OPENROUTER_API_KEY`,

then **Validate** each.

- Set `MODEL_OPUS`, `MODEL_SONNET`, `MODEL_HAIKU`, and the fallback `MODEL` to the slugs

in `free-clause-code.env`. **Validate** each slug — if one 404s, the catalog moved; swap it for whatever that provider currently lists. Take the **structure** (capable / fast / fallback) as fixed, not my exact strings.

- **Apply**. Restart the server if it asks.
- Run `fcc-clause` to launch Claude Code through the proxy.

The one test that matters

Before trusting any of this, point `fcc` at a **single local Ollama model** (or one free provider) and have it complete one real multi-step task on a **throwaway git repo**. Watch whether it finishes a tool chain without flailing or leaving a half-edited mess. That single result tells you if *free-model *execution** is good enough to rely on. Only then expand to per-tier routing and your real repos.

Why fallback matters

Free tiers throw 429s mid-task. With a coding agent that has write access, an aborted run can leave a repo half-edited — worse than no run. A fallback `MODEL` on a **different** provider means a NIM limit rolls over to OpenRouter instead of stopping cold. Always work on a clean tree so a bad run is one `git reset --hard` away.

config/free-clause-code.env

```
# -----
# free-clause-code - recommended per-tier routing (free-only)
# -----
# This is a REFERENCE. You don't hand-edit free-clause-code's managed file -
# you enter these values in its Admin UI (Validate -> Apply). See config/README.md.
#
# Strategy: capable model on the tiers that think + make final edits,
# fast model on the Haiku tier for subagents/grunt work,
# a DIFFERENT provider as fallback so one rate-limit doesn't kill a session.

# Architect / hard reasoning + main coding (capable):
MODEL_OPUS=nvidia_nim/nvidia/nemotron-3-super-120b-a12b
MODEL_SONNET=nvidia_nim/nvidia/nemotron-3-super-120b-a12b

# Subagents + grunt work (fast):
MODEL_HAIKU=groq/llama-3.3-70b-versatile

# Fallback when a tier returns 429 - keep it on a provider different from the tiers above:
MODEL=open_router/openrouter/free

# Keys you'll paste into the Admin UI (free to obtain):
#   NVIDIA_NIM_API_KEY   -> https://build.nvidia.com/settings/api-keys
#   GROQ_API_KEY         -> https://console.groq.com/keys
#   OPENROUTER_API_KEY   -> https://openrouter.ai/keys

# Leave the 190k auto-compact window that fcc ships with ON - it stretches free quota.
```

```
# -- Local opt-in (if you ever have the RAM) --  
# Swap any tier to a local backend, e.g.:  
#   MODEL_HAIKU=ollama/qwen2.5-coder  
# free-claude-code already ships Ollama / LM Studio / llama.cpp - no extra code.
```

6. Full Source

Complete snapshot — enough to reconstruct or hand to any AI for full context.

Python

triad/config.py

```
"""Central configuration - resolved LAZILY from the environment.

The earlier version snapshotted os.environ into a module-level dict at import time, so
any `TRIAD_*` override that arrived later (from `.env`, or set after import) was silently
ignored - and the bug came back the moment anything imported config before the .env load.
Reading the environment at agent-construction time instead makes import order irrelevant.

Model IDs change constantly and free catalogs drift fast, so everything here is
env-overridable; validate any free slug before trusting it. `base_url` (optional) points an
agent at an OpenAI- or Anthropic-compatible gateway instead of the provider's own endpoint -
that's the seam for free/local models.
"""
import os

# Static spec only: which env vars to read + the fallback default. No env reads here -
# resolve() does them on demand. (env var, default) resolved at construction.
_SPECS = {
    # ---- paid frontier roster ----
    "chatgpt": {"key_env": "OPENAI_API_KEY",
                "model_env": "TRIAD_OPENAI_MODEL", "model_default": "gpt-4.1"},
    "claude": {"key_env": "ANTHROPIC_API_KEY",
               "model_env": "TRIAD_CLAUDE_MODEL", "model_default": "claude-sonnet-4-6",
               # Set TRIAD_CLAUDE_BASE_URL to free-claude-code (e.g. http://localhost:8082) to run
               # the Claude slot on a FREE model - fcc speaks the Anthropic API, so nothing else
               # changes. fcc routes by the model's *tier*, so TRIAD_CLAUDE_MODEL just selects the
               # tier (claude-sonnet-4-6 -> fcc's MODEL_SONNET); the free model is set in fcc.
               "base_url_env": "TRIAD_CLAUDE_BASE_URL", # no default -> paid Anthropic unless set
    "gemini": {"key_env": "GEMINI_API_KEY",
               "model_env": "TRIAD_GEMINI_MODEL", "model_default": "gemini-2.5-pro"},

    # ---- free-cloud roster (all OpenAI-compatible; differ only by base_url + model) ----
    # Defaults span THREE model lineages on purpose - Llama / GPT-OSS / Qwen. Three finetunes
    # of one base make correlated errors and hollow out aggregation, so keep lineages distinct.
    "groq": {"key_env": "GROQ_API_KEY", # Llama lineage
             "model_env": "TRIAD_GROQ_MODEL", "model_default": "llama-3.3-70b-versatile",
             "base_url_env": "TRIAD_GROQ_BASE_URL",
             "base_url_default": "https://api.groq.com/openai/v1"},
    "openrouter": {"key_env": "OPENROUTER_API_KEY", # GPT-OSS lineage (free; catalog drifts -
                    verify)
                  "model_env": "TRIAD_OPENROUTER_MODEL",
                  "model_default": "openai/gpt-oss-120b:free",
                  "base_url_env": "TRIAD_OPENROUTER_BASE_URL",
                  "base_url_default": "https://openrouter.ai/api/v1"},
    "nim": {"key_env": "NVIDIA_NIM_API_KEY", # Qwen lineage
            "model_env": "TRIAD_NIM_MODEL", "model_default":
    "qwen/qwen2.5-coder-32b-instruct",
            "base_url_env": "TRIAD_NIM_BASE_URL",
            "base_url_default": "https://integrate.api.nvidia.com/v1"},
}

AGENTS = tuple(_SPECS) # all configurable agent names, in order

def resolve(name: str) -> dict:
    """Resolve one agent's config from the CURRENT environment.

    Called at agent construction (not import), so `.env` and exported overrides always win
    regardless of import order. Returns {"model", "key_env", "base_url"}.
    """
    s = _SPECS[name]
    base_url = None
    if "base_url_env" in s:
        base_url = os.environ.get(s["base_url_env"], s.get("base_url_default"))
    return {
        "model": os.environ.get(s["model_env"], s["model_default"]),
        "key_env": s["key_env"],
        "base_url": base_url,
    }
```

```
def max_tokens() -> int:
    """Max output tokens per turn (Anthropic requires it; others honor it). Read lazily."""
    return int(os.environ.get("TRIAD_MAX_TOKENS", "4096"))
```

triad/agents.py

"""Provider-backed agents.

Each agent wraps one provider's async SDK behind a common interface:

```
* stream(user_msg)      -> async generator of text deltas (keeps conversation history)
* complete(user_msg)    -> full string (keeps conversation history)
* stream_raw(messages) -> async generator (ephemeral, no history mutation)
* complete_raw(messages)-> full string (ephemeral)
```

SDKs are imported lazily so the package loads even when a provider isn't installed, and a missing key just disables that agent instead of crashing the app.

```
"""
from __future__ import annotations

import inspect
import os
from typing import AsyncIterator, Dict, List, Optional

from . import config

Message = Dict[str, str] # {"role": "user"|"assistant", "content": str}

class Agent:
    name: str = "agent"
    label: str = "Agent"
    color: str = "white"

    def __init__(self) -> None:
        cfg = config.resolve(self.name) # reads env now, not at import - overrides always apply
        self.model: str = cfg["model"]
        self.api_key: str = os.environ.get(cfg["key_env"], "").strip()
        self.base_url: Optional[str] = cfg.get("base_url")
        self.system_prompt: str = ""
        self.history: List[Message] = []
        self._client = None

    # ----- meta
    @property
    def available(self) -> bool:
        if self.api_key:
            return True
        # A local/self-hosted gateway (free-claude-code, Ollama, LM Studio) needs no key.
        return bool(self.base_url) and any(
            h in self.base_url for h in ("localhost", "127.0.0.1", "0.0.0.0", "::1"))

    def set_system(self, text: str) -> None:
        self.system_prompt = text.strip()

    def add_skill(self, body: str) -> None:
        body = body.strip()
        self.system_prompt = f"{self.system_prompt}\n\n{body}".strip() if self.system_prompt else body

    def clear(self) -> None:
        self.history = []

    # ----- persistent chat
    async def stream(self, user_msg: str) -> AsyncIterator[str]:
        """Stream a reply and append the full exchange to this agent's history."""
        self.history.append({"role": "user", "content": user_msg})
        full = ""
        async for delta in self._provider_stream(self.history, self.system_prompt):
            full += delta
            yield delta
        self.history.append({"role": "assistant", "content": full})

    async def complete(self, user_msg: str) -> str:
        out = ""
        async for d in self.stream(user_msg):
            out += d
```

```

        return out

# ----- ephemeral (orchestration)
async def stream_raw(
    self, messages: List[Message], system: Optional[str] = None
) -> AsyncIterator[str]:
    sys_prompt = system if system is not None else self.system_prompt
    async for delta in self._provider_stream(messages, sys_prompt):
        yield delta

async def complete_raw(self, messages: List[Message], system: Optional[str] = None) -> str:
    out = ""
    async for d in self.stream_raw(messages, system):
        out += d
    return out

# ----- provider hook
async def _provider_stream(self, messages: List[Message], system: str) -> AsyncIterator[str]:
    raise NotImplementedError
    yield # pragma: no cover (makes this an async generator)

# -----
class OpenAICompatibleAgent(Agent):
    """Any OpenAI-compatible chat endpoint. OpenAI itself uses the default base_url;
    Groq / OpenRouter / NIM / a local gateway just set `base_url` in config. Subclasses
    only need name/label/color - model, key_env, and base_url all come from config."""

    def _obj(self):
        if self._client is None:
            try:
                from openai import AsyncOpenAI
            except ImportError as e:
                raise RuntimeError("OpenAI SDK missing - run: pip install openai") from e
            kwargs = {"api_key": self.api_key or "not-needed"}
            if self.base_url:
                kwargs["base_url"] = self.base_url
            self._client = AsyncOpenAI(**kwargs)
        return self._client

    async def _provider_stream(self, messages, system):
        client = self._obj()
        msgs: List[Message] = []
        if system:
            msgs.append({"role": "system", "content": system})
        msgs.extend(messages)
        stream = await client.chat.completions.create(
            model=self.model, messages=msgs, stream=True, max_tokens=config.max_tokens()
        )
        async for chunk in stream:
            if chunk.choices:
                delta = chunk.choices[0].delta
                if delta and delta.content:
                    yield delta.content

class OpenAIAgent(OpenAICompatibleAgent):
    name, label, color = "chatgpt", "ChatGPT", "green"

# Free-cloud roster - all OpenAI-compatible, differing only by base_url + model.
# Keep the model lineages DISTINCT (Llama / GPT-OSS / Qwen): three finetunes of one
# base make correlated errors and then agree, which hollows out any aggregation.
class GroqAgent(OpenAICompatibleAgent):
    name, label, color = "groq", "Groq", "magenta"

class OpenRouterAgent(OpenAICompatibleAgent):
    name, label, color = "openrouter", "OpenRouter", "bright_cyan"

class NIMAgent(OpenAICompatibleAgent):
    name, label, color = "nim", "NIM", "bright_green"

# -----
class ClaudeAgent(Agent):
    name, label, color = "claude", "Claude", "dark_orange3"

```

```

def _obj(self):
    if self._client is None:
        try:
            from anthropic import AsyncAnthropic
        except ImportError as e:
            raise RuntimeError("Anthropic SDK missing - run: pip install anthropic") from e
        kwargs = {"api_key": self.api_key or "not-needed"}
        # TRIAD_CLAUDE_BASE_URL (e.g. free-claude-code on http://localhost:8082) makes the
        # Claude slot run on a free model; fcc speaks the Anthropic API so nothing else changes.
        # Passed explicitly (not via global ANTHROPIC_BASE_URL) so it can't hijack other
clients.
        if self.base_url:
            kwargs["base_url"] = self.base_url
        self._client = AsyncAnthropic(**kwargs)
    return self._client

async def _provider_stream(self, messages, system):
    client = self._obj()
    kwargs = dict(model=self.model, max_tokens=config.max_tokens(), messages=messages)
    if system:
        kwargs["system"] = system
    async with client.messages.stream(**kwargs) as stream:
        async for text in stream.text_stream:
            yield text

# -----
class GeminiAgent(Agent):
    name, label, color = "gemini", "Gemini", "blue"

    def _obj(self):
        if self._client is None:
            try:
                from google import genai
            except ImportError as e:
                raise RuntimeError("Google GenAI SDK missing - run: pip install google-genai") from e
            self._client = genai.Client(api_key=self.api_key)
        return self._client

    @staticmethod
    def _to_contents(messages: List[Message]):
        contents = []
        for m in messages:
            role = "model" if m["role"] == "assistant" else "user"
            contents.append({"role": role, "parts": [{"text": m["content"]}]}))
        return contents

    async def _provider_stream(self, messages, system):
        client = self._obj()
        from google.genai import types

        cfg = types.GenerateContentConfig(
            system_instruction=system or None,
            max_output_tokens=config.max_tokens(),
        )
        maybe = client.aio.models.generate_content_stream(
            model=self.model, contents=self._to_contents(messages), config=cfg
        )
        # SDK surface has shifted between versions: it may return a coroutine
        # (await -> async iterator) or an async iterator directly. Handle both.
        stream = await maybe if inspect.isawaitable(maybe) else maybe
        async for chunk in stream:
            text = getattr(chunk, "text", None)
            if text:
                yield text

# -----
PAID_AGENTS = (OpenAIAgent, ClaudeAgent, GeminiAgent)
FREE_AGENTS = (GroqAgent, OpenRouterAgent, NIMAgent)
ROSTERS = {"paid": PAID_AGENTS, "free": FREE_AGENTS, "all": PAID_AGENTS + FREE_AGENTS}

def build_agents(roster: str = "paid") -> List[Agent]:
    """Return the agents in `roster` whose key (or local base_url) is present, in order.

    roster: 'paid' (OpenAI/Anthropic/Gemini), 'free' (Groq/OpenRouter/NIM, distinct
    lineages), or 'all'. The free roster is *substrate*, not a validated capability:

```

```

three free chat opinions make correlated errors and then agree, so they need an
executable verifier (the generate-verify-select work) before any answer is trusted.
"""
agents: List[Agent] = []
for cls in ROSTERS.get(roster, PAID_AGENTS):
    a = cls()
    if a.available:
        agents.append(a)
return agents

```

triad/skills.py

"""Skill files: markdown with a tiny YAML-ish frontmatter block.

```

---
name: code-reviewer
description: Hunts for bugs, edge cases, and security issues
agents: [gemini]      # who it targets by default; "all" = everyone
---
<instructions the agent receives as part of its system prompt>

```

The frontmatter parser is intentionally minimal so there's no PyYAML dependency.

```

"""
from __future__ import annotations

import glob
import os
from dataclasses import dataclass, field
from typing import Dict, List, Tuple

@dataclass
class Skill:
    name: str
    description: str = ""
    agents: List[str] = field(default_factory=lambda: ["all"])
    body: str = ""

def _parse_frontmatter(text: str) -> Tuple[Dict[str, str], str]:
    meta: Dict[str, str] = {}
    body = text
    if text.startswith("---"):
        end = text.find("\n---", 3)
        if end != -1:
            block = text[3:end].strip()
            body = text[end + 4 :].lstrip("\n")
            for line in block.splitlines():
                if ":" in line:
                    k, v = line.split(":", 1)
                    meta[k.strip()] = v.strip()
    return meta, body

def load_skills(path: str) -> Dict[str, Skill]:
    skills: Dict[str, Skill] = {}
    if not os.path.isdir(path):
        return skills
    for fp in sorted(glob.glob(os.path.join(path, "*.md"))):
        with open(fp, encoding="utf-8") as fh:
            meta, body = _parse_frontmatter(fh.read())
            name = meta.get("name") or os.path.splitext(os.path.basename(fp))[0]
            raw = meta.get("agents", "all").strip().strip("[]")
            agents = [a.strip() for a in raw.split(",") if a.strip()] or ["all"]
            skills[name] = Skill(
                name=name,
                description=meta.get("description", ""),
                agents=agents,
                body=body.strip(),
            )
    return skills

```

triad/ui.py

"""Terminal rendering with `rich`.

Two display primitives:

```

    * live_parallel - N agents stream simultaneously into side-by-side panels.
    * live_single   - one agent streams into a full-width panel (used for relay/council steps).
"""
from __future__ import annotations

import asyncio
import textwrap
from typing import Callable, Dict, List

from rich.columns import Columns
from rich.console import Console
from rich.live import Live
from rich.panel import Panel
from rich.text import Text

def _fit(text: str, width: int, height: int) -> str:
    """Wrap to `width` and keep only the last `height` rows (so the latest text stays visible)."""
    rows: List[str] = []
    for line in text.split("\n"):
        if line == "":
            rows.append("")
        else:
            rows.extend(textwrap.wrap(line, width=max(8, width)) or [""])
    if len(rows) > height:
        rows = rows[-height:]
    return "\n".join(rows)

def _columns(console: Console, agents, buffers: Dict[str, str], statuses: Dict[str, str]):
    n = max(1, len(agents))
    col_w = max(26, console.width // n - 1)
    inner_w = col_w - 4
    height = max(8, console.height - 8)
    panels = []
    for a in agents:
        body = _fit(buffers.get(a.name, ""), inner_w, height)
        panels.append(
            Panel(
                Text(body),
                title=f"[bold]{a.label}[/bold]",
                subtitle=statuses.get(a.name, ""),
                border_style=a.color,
                width=col_w,
                height=height + 2,
                padding=(0, 1),
            )
        )
    return Columns(panels, equal=True, expand=True)

async def live_parallel(
    console: Console,
    agents,
    stream_factory: Callable, # stream_factory(agent) -> async iterator of text deltas
) -> Dict[str, str]:
    """Run every agent's stream concurrently, rendering all of them live."""
    buffers: Dict[str, str] = {a.name: "" for a in agents}
    statuses: Dict[str, str] = {a.name: "[dim]* thinking...[/dim]" for a in agents}

    async def run(agent):
        try:
            async for delta in stream_factory(agent):
                buffers[agent.name] += delta
                statuses[agent.name] = "[green][ok] done[/green]"
        except Exception as e: # one agent failing shouldn't kill the others
            statuses[agent.name] = "[red][x] error[/red]"
            buffers[agent.name] += f"\n\n[error] {type(e).__name__}: {e}"

    tasks = [asyncio.create_task(run(a)) for a in agents]
    try:
        with Live(_columns(console, agents, buffers, statuses), console=console,
            refresh_per_second=12, screen=False) as live:
            while not all(t.done() for t in tasks):
                live.update(_columns(console, agents, buffers, statuses))
                await asyncio.sleep(0.08)
            live.update(_columns(console, agents, buffers, statuses))
    finally:
        # On Ctrl-C / error, don't leave provider streams running in the background.

```

```

        for t in tasks:
            if not t.done():
                t.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)
    return dict(buffers)

async def live_single(console: Console, agent, stream_iter, title: str) -> str:
    """Stream a single agent into one full-width panel."""
    buffer = ""
    status = "[dim]* thinking...[/dim]"

    def render():
        height = max(6, console.height - 8)
        body = _fit(buffer, console.width - 4, height)
        return Panel(Text(body), title=f"[bold]{title}[/bold]", subtitle=status,
                     border_style=agent.color, padding=(0, 1))

    with Live(render(), console=console, refresh_per_second=12, screen=False) as live:
        try:
            async for delta in stream_iter:
                buffer += delta
                live.update(render())
                status = "[green][ok] done[/green]"
        except Exception as e:
            status = "[red][x] error[/red]"
            buffer += f"\n\n[error] {type(e).__name__}: {e}"
        live.update(render())
    return buffer

```

triad/orchestrator.py

"""Collaboration modes.

parallel - same task to everyone at once; independent answers, side by side.
 relay - agents work in sequence, each building on the running transcript.
 council - everyone answers, then a chair agent synthesizes the best single answer.

When `protocol` is on, relay/council pass compact structured handoffs (see protocol.py) instead of re-sending the whole transcript, and report how much context that saved.

```

"""
from __future__ import annotations

from typing import Dict, List, Optional

from rich.console import Console

from .agents import Agent
from .oracle import AbsentOracle, extract_code
from .protocol import PROTOCOL_INSTRUCTION, RefStore, compact_block, est_tokens
from .sandbox import Sandbox
from .ui import live_parallel, live_single

class Orchestrator:
    def __init__(self, agents: List[Agent], skills: Dict, console: Console,
                 mode: str = "parallel", chair: Optional[str] = None) -> None:
        self.agents = agents
        self.skills = skills
        self.console = console
        self.mode = mode
        self.chair = chair or (agents[0].name if agents else None)
        self.protocol = False
        self.refs = RefStore()
        self.transcript: List[str] = [] # chronological session log for /save (all modes)
        self.oracle = None # verify-mode pass condition; None -> unverified (selection only)

    def _get(self, name: str) -> Agent:
        for a in self.agents:
            if a.name == name:
                return a
        return self.agents[0]

    def _record(self, task: str, outputs: List) -> None:
        """Append one turn to the session transcript so /save captures every mode.

        relay/council stream via stream_raw and never touch agent.history, so without

```

```

this their output would be lost on save. outputs: list of (label, text).
"""
block = [f"## [{self.mode}] {task}", ""]
for label, text in outputs:
    block.append(f"### {label}")
    block.append((text or "").strip())
    block.append("")
self.transcript.append("\n".join(block).rstrip() + "\n")

def _savings(self, baseline_chars: int, actual_chars: int) -> None:
    """Report compaction of the context handed between agents (raw vs digest)."""
    base, act = est_tokens("x" * baseline_chars), est_tokens("x" * actual_chars)
    if base <= 0:
        return
    pct = 100 * (base - act) / base
    self.console.print(
        f"[dim]protocol: ~{act} tokens of handoff context vs ~{base} raw "
        f"(~{pct:.0f}% less passed between agents). Grows with longer chats.[/dim]"
    )

async def dispatch(self, task: str) -> None:
    if self.mode == "relay":
        await self.run_relay(task)
    elif self.mode == "council":
        await self.run_council(task)
    elif self.mode == "verify":
        await self.run_verify(task)
    else:
        await self.run_parallel(task)

# ----- modes
async def run_parallel(self, task: str) -> None:
    # Parallel has no handoff to compress, so protocol doesn't apply here.
    buffers = await live_parallel(self.console, self.agents, lambda a: a.stream(task))
    self._record(task, [(a.label, buffers.get(a.name, "")) for a in self.agents])

async def run_relay(self, task: str) -> None:
    raw_accum = "" # full outputs concatenated (what a full transcript passes)
    note_accum = "" # compact notes concatenated (what protocol passes)
    baseline_chars = actual_chars = 0
    outputs = [] # (label, text) per step, for the session transcript

    for i, a in enumerate(self.agents):
        if self.protocol:
            if i == 0:
                prompt = f"TASK:\n{task}\n\n{PROTOCOL_INSTRUCTION}"
            else:
                prompt = (
                    f"TASK:\n{task}\n\nPRIOR WORK (compact handoffs):\n{note_accum.strip()}\n\n"
                    f"Add your contribution, building on the above.\n\n{PROTOCOL_INSTRUCTION}"
                )
        else:
            if i == 0:
                prompt = task
            else:
                prompt = (
                    "You are collaborating with other AI agents on a shared task.\n\n"
                    f"TASK:\n{task}\n\nWORK SO FAR (from other"
                    agents):\n{raw_accum.strip()}\n\n"
                    "Add your contribution: build on it, fix mistakes, fill gaps. "
                    "Don't just repeat what's already there."
                )

        if i > 0: # context re-passed at this hop, raw vs compact
            baseline_chars += len(raw_accum)
            actual_chars += len(note_accum)

        text = await live_single(
            self.console, a, a.stream_raw([{"role": "user", "content": prompt}]),
            f"{a.label} - relay step {i + 1}/{len(self.agents)}",
        )

        self.refs.put(f"relay/{i + 1}", text)
        outputs.append((a.label, text))
        raw_accum += f"\n\n## {a.label}\n{text}"
        if self.protocol:
            note_accum += "\n\n" + compact_block(text, a.label)

    self._record(task, outputs)

```

```

        if self.protocol:
            self._savings(baseline_chars, actual_chars)

    async def run_council(self, task: str) -> None:
        self.console.rule("[bold]Round 1 - independent answers[/bold]")
        instr = f"[task]\n\n[PROTOCOL_INSTRUCTION]" if self.protocol else task
        answers = await live_parallel(
            self.console, self.agents,
            lambda a: a.stream_raw([{"role": "user", "content": instr}]),
        )

        if self.protocol:
            blocks, baseline_chars, actual_chars = [], 0, 0
            for i, a in enumerate(self.agents):
                self.refs.put(f"council/{a.name}", answers[a.name])
                block = compact_block(answers[a.name], f"Response {chr(65 + i)}")
                blocks.append(block)
                actual_chars += len(block)
                baseline_chars += len(answers[a.name])
            body = "\n\n".join(blocks)
        else:
            body = "\n\n".join(
                f"### Response {chr(65 + i)}\n{answers[a.name]}" for i, a in enumerate(self.agents)
            )

        chair = self._get(self.chair)
        self.console.rule(f"[bold]Round 2 - {chair.label} synthesizes[/bold]")
        synth = (
            "Several AI agents independently answered the task below. As the chair, "
            "produce the single best final answer: merge their strengths, resolve "
            "disagreements, correct errors, and flag any important dissent.\n\n"
            f"TASK:\n{task}\n\nRESPONSES:\n{body}"
        )
        synth_text = await live_single(
            self.console, chair, chair.stream_raw([{"role": "user", "content": synth}]),
            f"{chair.label} - synthesis",
        )
        outputs = [(a.label, answers[a.name]) for a in self.agents]
        outputs.append((f"{chair.label} - synthesis", synth_text))
        self._record(task, outputs)
        if self.protocol:
            self._savings(baseline_chars, actual_chars)

# ----- verify
@staticmethod
def _gen_prompt(task: str, fail: Optional[str] = None) -> str:
    p = (f"TASK:\n{task}\n\nReturn a complete, self-contained solution as ONE fenced code "
        "block (``), and nothing outside the block.")
    if fail:
        p += (f"\n\nYour previous attempt FAILED verification:\n{fail}\n\nFix it and return the "
            "full corrected solution as one code block.")
    return p

    async def run_verify(self, task: str, rounds: int = 3) -> None:
        """generate-verify-select. N candidates -> independent oracle -> pick a passer, else
        critique-revise (bounded). With NO independent oracle: generate and report UNVERIFIED
        (selection only) - never a self-grade, never a fabricated pass."""
        oracle = self.oracle or AbsentOracle()
        sandbox = Sandbox()
        self.console.print(f"[dim]verify: oracle = {oracle.describe()}[/dim]")
        self.console.print(f"[dim]verify: sandbox = {sandbox.note}[/dim]")

        # ---- no independent oracle: selection only ----
        if not oracle.independent:
            self.console.rule("[bold]verify - generate (no oracle)[/bold]")
            gen = await live_parallel(
                self.console, self.agents,
                lambda a: a.stream_raw([{"role": "user", "content": self._gen_prompt(task)}])
            )
            outputs = [(a.label, gen.get(a.name, "")) for a in self.agents]
            outputs.append(("verify", "UNVERIFIED - selection only (no oracle ran; nothing
            executed/checked)"))
            self._record(task, outputs)
            self.console.print("[yellow]■ UNVERIFIED - no oracle. Selection only: review the
            candidates "
                                "above and choose; nothing was executed or checked. Set one with "
                                "[bold]/oracle <cmd>[/bold] (e.g. /oracle \"pytest -q\").[/yellow]")

            return

```

```

# ---- oracle present: generate -> verify -> select -> critique-revise ----
last_fail: Dict[str, str] = {}
for rnd in range(1, rounds + 1):
    self.console.rule(f"[bold]verify - round {rnd}/{rounds}[/bold]")
    gen = await live_parallel(
        self.console, self.agents,
        lambda a: a.stream_raw([{"role": "user",
                                "content": self._gen_prompt(task,
last_fail.get(a.name))}]))
    results = []
    for a in self.agents:
        v = oracle.check(gen.get(a.name, ""), sandbox)
        results.append((a, extract_code(gen.get(a.name, "")), v))
        mark = {"pass": "[green][ok] pass[/green]", "fail": "[red][x] fail[/red]",
                "unverified": "[yellow]? unverified[/yellow]"}[v.status]
        self.console.print(f" {a.label}: {mark} [dim]{v.detail}[/dim]")

    passers = [(a, code, v) for (a, code, v) in results if v.passed]
    if passers:
        a, code, v = passers[0]
        extra = "" if len(passers) == 1 else f" ({len(passers)} passed; picked {a.label},
first in order)"
        self.console.rule(f"[bold green]VERIFIED - {a.label} passed{extra}[/bold green]")
        self.console.print(code)
        self._record(task, [(a.label, code),
                            ("verify", f"PASSED - {a.label} round {rnd} ({v.detail})")])
        return

    last_fail = {a.name: v.detail for (a, code, v) in results}
    if rnd == rounds:
        self.console.rule("[bold red]verify - no candidate passed[/bold red]")
        self.console.print(f"[red]0/{len(self.agents)} passed after {rounds} round(s). "
                            "Candidates recorded; nothing verified.[/red]")
        outputs = [(a.label, code) for (a, code, v) in results]
        outputs.append(("verify", f"FAILED - 0/{len(self.agents)} passed after {rounds}
rounds"))
        self._record(task, outputs)

```

triad/protocol.py

"""Compact inter-agent handoff protocol.

The point: in relay/council, agents currently re-send the whole growing transcript on every hop. That's the real token waste. This module gives a terse, in-distribution (so capability holds) and human-readable (so you keep oversight) handoff format, plus a reference store so full outputs live somewhere addressable while only compact digests get passed forward.

Format agents are asked to emit:

```

@goal <one line>
@find
- <terse point>
- <terse point>
@conf <0.0-1.0>
@next <handoff note / 'done'>
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, List

PROTOCOL_INSTRUCTION = (
    "Respond in this compact handoff format, nothing else:\n"
    "@goal <one line: what you're solving>\n"
    "@find\n"
    "- <terse finding or contribution>\n"
    "- <terse finding>\n"
    "@conf <your confidence, 0.0-1.0>\n"
    "@next <handoff: what's open / who should act / 'done'>\n"
    "Be dense. No preamble, no restating the question."
)

```

```

def est_tokens(text: str) -> int:
    """Rough token count (~4 chars/token). Fine for relative before/after comparison."""
    return max(1, len(text) // 4)

```

```

@dataclass
class Note:
    raw: str
    goal: str = ""
    find: List[str] = field(default_factory=list)
    conf: str = ""
    nxt: str = ""

def parse_note(text: str) -> Note:
    note = Note(raw=text)
    section = None
    for line in text.splitlines():
        s = line.strip()
        low = s.lower()
        if low.startswith("@goal"):
            note.goal = s[5:].strip(": ").strip()
            section = None
        elif low.startswith("@find"):
            section = "find"
        elif low.startswith("@conf"):
            note.conf = s[5:].strip(": ").strip()
            section = None
        elif low.startswith("@next"):
            note.nxt = s[5:].strip(": ").strip()
            section = None
        elif section == "find" and s.startswith(("-", "*", "")):
            note.find.append(s.lstrip("-* ").strip())
    return note

def compact_block(text: str, label: str, max_chars: int = 700) -> str:
    """A short, forward-passable digest of one agent's output.

    If the agent obeyed the protocol, re-emit the essential lines. Otherwise fall back
    to a truncated digest so an off-format reply still doesn't blow up the next prompt.
    """
    note = parse_note(text)
    if note.find or note.conf or note.nxt:
        parts = [f"## {label}"]
        if note.find:
            parts.append("@find")
            parts.extend(f"- {f}" for f in note.find)
        if note.conf:
            parts.append(f"@conf {note.conf}")
        if note.nxt:
            parts.append(f"@next {note.nxt}")
        return "\n".join(parts)
    # fallback: trim raw
    trimmed = text.strip()
    if len(trimmed) > max_chars:
        trimmed = trimmed[:max_chars].rsplit(" ", 1)[0] + " ..."
    return f"## {label}\n{trimmed}"

class RefStore:
    """Holds full outputs under addressable keys so handoffs can pass a pointer
    (ref:key) and the human transcript can still recover everything."""

    def __init__(self) -> None:
        self._d: Dict[str, str] = {}

    def put(self, key: str, value: str) -> str:
        self._d[key] = value
        return f"ref:{key}"

    def get(self, key: str) -> str:
        return self._d.get(key, "")

    def clear(self) -> None:
        self._d.clear()

```

triad/vault.py

"""Obsidian-compatible vault: persist a triad session as a living knowledge graph of markdown notes with [[wikilinks]], so the folder opens straight into Obsidian's graph view.

Layout (open the folder itself as a vault):

```

index.md          hub - links every agent + topic + checkpoint note
agents/<name>.md   one living note per agent (role + a dated Log, appended each turn)
topics/<topic>.md  seeded project concepts, cross-linked so the graph isn't empty
checkpoints/<...>.md session checkpoints

```

Wikilinks resolve by basename across the whole vault, so link text == filename (spaces are fine). Everything here is plain files: Obsidian watches the folder and shows new notes live.

```

"""
from __future__ import annotations

import datetime
import re
from pathlib import Path
from typing import Dict, List, Optional, Tuple

# ---- seeded project concepts (the graph spine). Bodies cross-link via [[wikilinks]]. ----
TOPICS: Dict[str, str] = {
    "free-cloud default": (
        "Default to **free cloud** models, not local. The target user has no budget and a "
        "MacBook Air, so 'just run it locally' is the privilege answer - free cloud routing is "
        "the point, local is an opt-in.\n\n"
        "Related: [[cross-provider decorrelation]], [[layer 2 free-claude-code]], [[generate-verify-
select]]."
    ),
    "cross-provider decorrelation": (
        "Aggregating free models only helps if their errors are **independent**. Three vendors "
        "serving finetunes of one base (all Llama, say) make *correlated* errors and then agree - "
        "pseudo-diversity. Pick distinct lineages (Llama / GPT-OSS / Qwen).\n\n"
        "Related: [[free-cloud default]], [[generate-verify-select]]."
    ),
    "generate-verify-select": (
        "The Stage 2 primitive for verifiable work: generate N diverse candidates -> run an "
        "***oracle** -> keep the ones that pass -> feed failures back to revise. Best-of-N against an "
        "
        "executable check, not a discussion (synthesis would blend correlated-wrong answers).\n\n"
        "Related: [[oracle independence]], [[execution sandbox]], [[cross-provider decorrelation]]."
    ),
    "oracle independence": (
        "generate-verify-select only works if the verifier isn't captured by the generator. If the "
        "free models write both the code and its tests, they'll write tests that pass their own
wrong "
        "code. The oracle must be independent: user tests, a separate test-author step, or ground-
truth "
        "execution.\n\n"
        "Related: [[generate-verify-select]], [[execution sandbox]]."
    ),
    "execution sandbox": (
        "Stage 2 runs free-model-***generated** code, so 'never give cheap models something you can't "
        "
        "`git reset` isn't enough - you need a real sandbox (subprocess/container, no host FS, no "
        "network, time/memory limits). Never on the host.\n\n"
        "Related: [[generate-verify-select]], [[oracle independence]]."
    ),
    "layer 1 triad": (
        "This repo: a terminal orchestrator running ChatGPT/Claude/Gemini (or a free roster) in "
        "parallel / relay / council. Paid frontier APIs by default; chat-only, never edits
files.\n\n"
        "Related: [[layer 2 free-claude-code]], [[layer 3 sidecar]], [[free-cloud default]]."
    ),
    "layer 2 free-claude-code": (
        "Separate tool: a proxy that drives a real file-editing coding agent on **free/local**
models. "
        "Anthropic-shaped (`v1/messages` on localhost:8082), so triad's Claude slot can run free "
        "through it via `TRIAD_CLAUDE_BASE_URL`.\n\n"
        "Related: [[layer 1 triad]], [[layer 3 sidecar]]."
    ),
    "layer 3 sidecar": (
        "Separate tool: cheap models as **advisory-only** reviewers - second opinions, no file
edits, "
        "no final calls.\n\n"
        "Related: [[layer 1 triad]], [[layer 2 free-claude-code]]."
    ),
}

# Which topics each agent's living note links to (gives agent<=>topic edges in the graph).
_AGENT_FOCUS: Dict[str, List[str]] = {
    "chatgpt": ["generate-verify-select", "execution sandbox"],
    "claude": ["oracle independence", "free-cloud default"],
    "gemini": ["cross-provider decorrelation", "layer 2 free-claude-code"],
}

```

```

    "groq": ["free-cloud default", "cross-provider decorrelation"],
    "openrouter": ["cross-provider decorrelation", "generate-verify-select"],
    "nim": ["free-cloud default", "execution sandbox"],
}
_DEFAULT_FOCUS = ["generate-verify-select", "free-cloud default"]

_LINK_RE = re.compile(r"\\[([^\]|#|#+)]") # capture link target, ignoring |alias and #heading

def _today() -> str:
    return datetime.date.today().isoformat()

def _write(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(text.rstrip() + "\n", encoding="utf-8")

def ensure_dirs(vault: Path) -> None:
    for sub in ("agents", "topics", "checkpoints"):
        (vault / sub).mkdir(parents=True, exist_ok=True)

def linkify(text: str) -> str:
    """Wrap the first mention of any known topic name in [...] so summaries form edges."""
    for name in sorted(TOPICS, key=len, reverse=True): # longest first: avoid nested matches
        if f"[{name}]" in text:
            continue
        m = re.search(re.escape(name), text, re.IGNORECASE)
        if m:
            text = text[:m.start()] + f"[{name}]" + text[m.end():]
    return text

def _summarize(text: str, limit: int = 240) -> str:
    s = " ".join(text.split())
    if len(s) <= limit:
        return s
    return s[:limit].rsplit(" ", 1)[0] + "..." # trim to a word boundary only when truncating

# ----- seeding
def seed_topics(vault: Path) -> int:
    """Write the concept notes (only if missing, so edits survive). Returns count written."""
    n = 0
    for name, body in TOPICS.items():
        p = vault / "topics" / f"{name}.md"
        if not p.exists():
            _write(p, f"# {name}\n\n{body}")
            n += 1
    return n

def seed_agent_note(vault: Path, name: str, label: str, model: str) -> None:
    """Create a living note for one agent if it doesn't exist yet."""
    p = vault / "agents" / f"{name}.md"
    if p.exists():
        return
    focus = _AGENT_FOCUS.get(name, _DEFAULT_FOCUS)
    links = ", ".join(f"[{t}]" for t in focus)
    body = (
        f"# {label}\n\n"
        f"Living note for the **{label}** agent (model: `{model}`). Part of the [[index|triad]] "
        f"roster - see [[layer 1 triad]]. Focus: {links}.\n\n"
        f"## Log\n"
        f"- {_today()} - joined the session; focus on {links}.\n"
    )
    _write(p, body)

def _list_notes(vault: Path, sub: str) -> List[str]:
    d = vault / sub
    return sorted(p.stem for p in d.glob("*.md")) if d.is_dir() else []

def seed_index(vault: Path) -> None:
    """(Re)write index.md, linking every agent, topic, and checkpoint that exists."""
    agents = _list_notes(vault, "agents")
    checkpoints = _list_notes(vault, "checkpoints")

```

```

lines = [
    "# triad - vault index",
    "",
    "Living knowledge graph for this triad project. Open this folder as an Obsidian vault "
    "(*Open folder as vault*) and hit the graph view (Cmd/Ctrl+G).",
    "",
    "## Agents",
    *([f"- [{a}]" for a in agents] or ["- _(none yet - run `triad --vault <dir>`)_"]),
    "",
    "## Topics",
    *[f"- [{t}]" for t in TOPICS],
    "",
    "## Checkpoints",
    *([f"- [{c}]" for c in checkpoints] or ["- _(none yet)_"]),
    "",
    f"_Updated {_today()} · triad --vault_",
]
_write(vault / "index.md", "\n".join(lines))

def open_vault(vault_dir: str, agents, skills=None) -> Path:
    """Create + seed the vault: topics, a living note per active agent, and the index."""
    vault = Path(vault_dir)
    ensure_dirs(vault)
    seed_topics(vault)
    for a in agents:
        seed_agent_note(vault, a.name, a.label, a.model)
    seed_index(vault)
    return vault

# ----- updating
def _last_turn_by_label(orch) -> Dict[str, str]:
    """Parse the most recent transcript block into {agent label: text} (works for every mode)."""
    transcript = getattr(orch, "transcript", None)
    if not transcript:
        return {}
    out: Dict[str, str] = {}
    cur: Optional[str] = None
    buf: List[str] = []
    for line in transcript[-1].splitlines():
        if line.startswith("### "):
            if cur is not None:
                out[cur] = "\n".join(buf).strip()
                cur, buf = line[4:].strip(), []
            elif line.startswith("## ["):
                continue
            elif cur is not None:
                buf.append(line)
    if cur is not None:
        out[cur] = "\n".join(buf).strip()
    return out

def remember(vault_dir: str, agents, orch) -> int:
    """Append each agent's latest contribution to its living note (linkified). Returns the
    number of agent notes updated. Safe to call every turn or from /remember."""
    vault = Path(vault_dir)
    ensure_dirs(vault)
    idx = len(getattr(orch, "transcript", []))
    if idx and getattr(orch, "_remembered_upto", 0) >= idx:
        return 0 # nothing new since the last write - don't duplicate the bullet
    by_label = _last_turn_by_label(orch)
    updated = 0
    for a in agents:
        seed_agent_note(vault, a.name, a.label, a.model) # ensure note exists (e.g. /keys added it)
        # Latest turn from the transcript (covers every mode); fall back to history if needed.
        text = by_label.get(a.label) or (
            a.history[-1]["content"] if a.history and a.history[-1]["role"] == "assistant" else ""
        )
        summary = _summarize(text)
        if not summary:
            continue
        bullet = f"- {_today()} ({orch.mode}) - {linkify(summary)}\n"
        with (vault / "agents" / f"{a.name}.md").open("a", encoding="utf-8") as fh:
            fh.write(bullet)
        updated += 1
    orch._remembered_upto = idx
    seed_index(vault)
    return updated

```

```

def checkpoint(vault_dir: str, title: str, body: str) -> Path:
    """Write a checkpoint note (linkified) and relink it from the index. Returns its path."""
    vault = Path(vault_dir)
    ensure_dirs(vault)
    name = f"{today()} {title}".strip()
    p = vault / "checkpoints" / f"{name}.md"
    _write(p, f"# {name}\n\n{linkify(body)}\n\nBack to [[index]].")
    seed_index(vault)
    return p

# ----- reading / checking
def resume_path(path: str) -> Path:
    """The file --seed-file should read: a vault dir resolves to its index.md."""
    p = Path(path)
    return p / "index.md" if p.is_dir() else p

def read_resume(path: str) -> str:
    """Read a vault's index.md (or a markdown file) back as resume context."""
    p = resume_path(path)
    return p.read_text(encoding="utf-8") if p.exists() else ""

def extract_links(text: str) -> List[str]:
    return [m.group(1).strip() for m in _LINK_RE.finditer(text)]

def check_links(vault_dir: str) -> Tuple[int, List[Tuple[str, str]]]:
    """Verify every [[wikilink]] resolves to a real note (by basename). Returns
    (total_links, orphans) where each orphan is (source_file, missing_target)."""
    vault = Path(vault_dir)
    names = {p.stem for p in vault.rglob("*.md")}
    total = 0
    orphans: List[Tuple[str, str]] = []
    for p in vault.rglob("*.md"):
        for target in extract_links(p.read_text(encoding="utf-8")):
            total += 1
            if target not in names:
                orphans.append((str(p.relative_to(vault)), target))
    return total, orphans

```

triad/sandbox.py

"""Run untrusted, model-generated code with as much isolation as the host actually offers.

Stage 2 (generate-verify-select) executes code that **free models wrote**. This module never runs it on the host unguarded, and it ALWAYS reports which isolation tier is active, so callers and users can see exactly what is and isn't contained.

Tiers (auto-selected, strongest first):

docker	real isolation - `--network none`, mem/cpu/pids caps, non-root, only a work dir mounted. Network AND filesystem isolated.
macos-seatbelt	macOS without Docker - `sandbox-exec` deny-network profile = a host-level no-net floor, plus a CPU rlimit. NETWORK is blocked; FILESYSTEM is NOT isolated (the code runs as you, with your file access).
subprocess	fallback - temp cwd + scrubbed env + CPU rlimit only. NOT a security boundary: no network block, no filesystem isolation. Reduced-isolation mode.

Honesty rules, baked in so nothing reads as safer than it is:

- * rlimits cap CPU/memory only - they do NOTHING for network or filesystem.
- * `Result.network\_blocked` is True only when the tier truly blocks it (docker / seatbelt) - a "best-effort" attempt is never reported as a block.
- * untrusted code with network access can exfiltrate; the active tier and its gaps ride on every Result and on `Sandbox.note`, so the UI surfaces them instead of proceeding silently.
- * `sandbox-exec` is DEPRECATED by Apple (works today). We never assume it enforces the profile - at startup we run a real network attempt under it and claim the no-net floor only if that attempt is actually blocked. If it isn't (or sandbox-exec is gone), we degrade to subprocess and say so loudly: a broken seatbelt fails as "no isolation, warned", never "network silently open while we report it shut".

"""

from __future__ import annotations

```

import os
import shutil
import subprocess

```

```

import sys
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional

try:
    import resource # POSIX only
except ImportError: # pragma: no cover (Windows)
    resource = None

# Allow normal operation but deny all network - keeps code runnable while removing the exfil path.
# (sandbox-exec is deprecated by Apple but still functional on current macOS.)
_SEATBELT_NO_NET = "(version 1)(allow default)(deny network*)"
_SAFE_PATH = "/usr/bin:/bin:/usr/sbin:/sbin"

# Run under the seatbelt profile to confirm it actually blocks the network. A real block raises
# PermissionError; anything else (connected, or some other error) means we can't claim a block.
_SEATBELT_PROBE = (
    "import socket\n"
    "try:\n"
    "    socket.create_connection(('1.1.1.1', 53), timeout=3); print('OPEN')\n"
    "except PermissionError: print('BLOCKED')\n"
    "except Exception as e: print('ERR:' + type(e).__name__)\n"
)

_detect_cache = None # (tier, degrade_note) memoized per process - host capability is stable

def _seatbelt_blocks_network() -> bool:
    """Actually exercise the seatbelt no-net profile; True only if a connection is really
    blocked."""
    try:
        p = subprocess.run(
            ["sandbox-exec", "-p", _SEATBELT_NO_NET, sys.executable, "-c", _SEATBELT_PROBE],
            capture_output=True, text=True, timeout=15)
        return p.returncode == 0 and p.stdout.strip() == "BLOCKED"
    except Exception:
        return False

def _detect_tier():
    """Strongest tier the host *actually* provides. Seatbelt is verified, never assumed."""
    global _detect_cache
    if _detect_cache is None:
        if _docker_available():
            _detect_cache = ("docker", "")
        elif sys.platform == "darwin" and shutil.which("sandbox-exec"):
            if _seatbelt_blocks_network():
                _detect_cache = ("macos-seatbelt", "")
            else:
                _detect_cache = ("subprocess",
                    " sandbox-exec is present but its deny-network profile did NOT block a test "
                    "connection (the deprecated mechanism may have changed) - degraded to
subprocess; "
                    "network is NOT blocked.")
        else:
            _detect_cache = ("subprocess", "")
    return _detect_cache

@dataclass
class Result:
    returncode: int
    stdout: str
    stderr: str
    timed_out: bool
    tier: str
    network_blocked: bool
    fs_isolated: bool
    note: str

    @property
    def ok(self) -> bool:
        return self.returncode == 0 and not self.timed_out

    @property
    def fully_isolated(self) -> bool:
        """True only when untrusted code is contained on BOTH axes (network and filesystem)."""
        return self.network_blocked and self.fs_isolated

```

```

def _docker_available() -> bool:
    if not shutil.which("docker"):
        return False
    try:
        return subprocess.run(["docker", "info"], capture_output=True, timeout=8).returncode == 0
    except Exception:
        return False

class Sandbox:
    """Pick the strongest isolation tier the host supports and run code under it."""

    def __init__(self, prefer: Optional[str] = None, image: str = "python:3.13-slim") -> None:
        self.image = image
        # prefer is an explicit override (mostly for tests); otherwise use the *verified* tier.
        self.tier, self._degrade = (prefer, "") if prefer else _detect_tier()
        self.network_blocked = self.tier in ("docker", "macos-seatbelt")
        self.fs_isolated = self.tier == "docker"
        self.note = self._describe()

    @property
    def fully_isolated(self) -> bool:
        """True only when untrusted code is contained on BOTH axes (network and filesystem)."""
        return self.network_blocked and self.fs_isolated

    def _describe(self) -> str:
        if self.tier == "docker":
            return "docker - network + filesystem isolated, mem/cpu/pids capped, non-root."
        if self.tier == "macos-seatbelt":
            return ("macos-seatbelt - NETWORK blocked (no-net floor, verified at startup) + CPU cap."
                    "FILESYSTEM NOT isolated: code runs as you. Install Docker for filesystem"
                    "isolation.")
        return ("subprocess - REDUCED ISOLATION: temp cwd + scrubbed env + CPU cap only."
                "NO network block, NO filesystem isolation; NOT a security boundary. Untrusted "
                "code can reach the network and your files. Install Docker for real isolation." +
                self._degrade)

    # ----- run
    def run(self, files: Dict[str, str], argv: List[str],
            timeout: int = 10, mem_mb: int = 512) -> Result:
        """Write `files` into a throwaway dir and run `argv` there under the active tier."""
        with tempfile.TemporaryDirectory(prefix="triad-sbx-") as tmp:
            root = Path(tmp)
            for name, content in files.items():
                fp = root / name
                fp.parent.mkdir(parents=True, exist_ok=True)
                fp.write_text(content, encoding="utf-8")
            if self.tier == "docker":
                cmd = self._docker_cmd(root, argv, mem_mb)
            elif self.tier == "macos-seatbelt":
                cmd = ["sandbox-exec", "-p", _SEATBELT_NO_NET, *argv]
            else:
                cmd = list(argv)
            return self._exec(cmd, root, timeout, mem_mb)

    def _docker_cmd(self, root: Path, argv: List[str], mem_mb: int) -> List[str]:
        return [
            "docker", "run", "--rm", "--network", "none",
            "--memory", f"{mem_mb}m", "--memory-swap", f"{mem_mb}m",
            "--cpus", "1", "--pids-limit", "128",
            "--user", f"{os.getuid()}:{os.getgid()}",
            "-v", f"{root}:/work", "-w", "/work",
            self.image, *argv,
        ]

    def _exec(self, cmd: List[str], cwd: Path, timeout: int, mem_mb: int) -> Result:
        # Scrubbed env so inherited secrets (API keys, etc.) never reach untrusted code; HOME -> tmp.
        env = {"PATH": _SAFE_PATH, "HOME": str(cwd), "TMPDIR": str(cwd), "LANG": "C.UTF-8"}

        def _limits(): # POSIX rlimits, set in the child before exec; docker enforces its own caps
            if resource is None:
                return
            for res, val in ((resource.RLIMIT_CPU, (max(1, timeout), max(1, timeout) + 1)),
                             (resource.RLIMIT_AS, (mem_mb * 1024 * 1024, * 2))): # macOS often
                resource.setrlimit(res, val)

        ignores AS
        try:
            resource.setrlimit(res, val)

```

```

        except Exception:
            pass

    preexec = None if (self.tier == "docker" or resource is None) else _limits
    try:
        p = subprocess.run(cmd, cwd=str(cwd), env=env, capture_output=True, text=True,
                           timeout=timeout + 2, preexec_fn=preexec)
        return self._result(p.returncode, p.stdout, p.stderr, False)
    except subprocess.TimeoutExpired as e:
        out = e.stdout if isinstance(e.stdout, str) else (e.stdout or b"").decode("utf-8",
        "replace")
        err = e.stderr if isinstance(e.stderr, str) else (e.stderr or b"").decode("utf-8",
        "replace")
        return self._result(124, out or "", (err or "") + "\n[sandbox] timed out", True)
    except FileNotFoundError as e:
        return self._result(127, "", f"[sandbox] cannot exec {cmd[0]!r}: {e}", False)

    def _result(self, rc: int, out: str, err: str, timed_out: bool) -> Result:
        return Result(rc, out, err, timed_out, self.tier,
                      self.network_blocked, self.fs_isolated, self.note)

def default_sandbox() -> Sandbox:
    return Sandbox()

```

triad/oracle.py

"""Independent verification for the `verify` mode (generate-verify-select).

An Oracle decides pass/fail for a candidate WITHOUT the candidate authoring its own pass condition - that's *oracle independence*. When no independent oracle is available, evaluation returns 'unverified' and the mode reports "selection only": it never fabricates a pass and never lets a candidate grade itself.

Oracles here:

- CommandOracle - run a user-authored command (e.g. `pytest -q`, or a `python3 -c "...` check) against the candidate's code in the sandbox. Independent: the command/tests are the user's; the candidate only supplies the code under test.
- AbsentOracle - no oracle configured -> every result is 'unverified' (selection only).

A separate test-author model (writes tests BEFORE seeing candidates) is a future independent oracle; the interface below is ready for it. See the vault note "open questions - verify mode".

```

"""
from __future__ import annotations

import re
from dataclasses import dataclass
from typing import Dict, List, Optional, Union

from .sandbox import Sandbox

_FENCE = re.compile(r"```[a-zA-Z0-9_+.\-]*\n(?:.*)```", re.DOTALL)

def extract_code(text: str) -> str:
    """The candidate's solution: the largest fenced code block, or the raw text if there's none."""
    blocks = _FENCE.findall(text or "")
    if blocks:
        return max(blocks, key=len).strip()
    return (text or "").strip()

@dataclass
class Verdict:
    status: str          # "pass" | "fail" | "unverified"
    detail: str = ""

    @property
    def passed(self) -> bool:
        return self.status == "pass"

    @property
    def verified(self) -> bool:
        return self.status in ("pass", "fail")

class Oracle:

```

```

name = "oracle"
independent = True          # the pass condition is NOT authored by the candidate being graded

def describe(self) -> str:
    return self.name

def check(self, candidate_text: str, sandbox: Sandbox) -> Verdict:
    raise NotImplementedError

class AbsentOracle(Oracle):
    """No independent oracle - selection only. Never a self-grade, never a fabricated pass."""
    name = "absent"
    independent = False

    def describe(self) -> str:
        return "none (unverified - selection only)"

    def check(self, candidate_text: str, sandbox: Sandbox) -> Verdict:
        return Verdict("unverified", "no oracle configured")

class CommandOracle(Oracle):
    """Run a user-authored command against the candidate's extracted code, in the sandbox.

    Independence holds by construction: `command` and `fixtures` come from the user (--oracle /
    /oracle), never from the candidate - the candidate only supplies the code under test. Pass iff
    the command exits 0 (and doesn't time out).
    """
    name = "command"

    def __init__(self, command: Union[str, List[str]], solution_name: str = "solution.py",
                 fixtures: Optional[Dict[str, str]] = None, timeout: int = 20):
        self.command = command
        self.solution_name = solution_name
        self.fixtures = fixtures or {}
        self.timeout = timeout

    def describe(self) -> str:
        cmd = self.command if isinstance(self.command, str) else " ".join(self.command)
        return f"command: {cmd}"

    def check(self, candidate_text: str, sandbox: Sandbox) -> Verdict:
        code = extract_code(candidate_text)
        if not code:
            return Verdict("fail", "candidate produced no code")
        files = {self.solution_name: code, **self.fixtures}
        argv = ["/bin/sh", "-c", self.command] if isinstance(self.command, str) else
        list(self.command)
        r = sandbox.run(files, argv, timeout=self.timeout)
        if r.timed_out:
            return Verdict("fail", f"timed out [{r.tier}]")
        msg = (r.stdout.strip() or r.stderr.strip())
        if len(msg) > 240:
            msg = msg[-240:]
        return Verdict("pass" if r.returncode == 0 else "fail", f"rc={r.returncode} [{r.tier}]
        {msg}".strip())

```

triad/dotenv.py

```

"""Minimal .env handling - parse, load into os.environ, and safely write keys.

No third-party dependency. Tolerates both `KEY=value` and `export KEY="value"`
styles so a hand-written or wizard-written file both work.
"""
from __future__ import annotations

import os
from pathlib import Path
from typing import Dict, Iterable, Union

PathLike = Union[str, Path]

def _strip_quotes(v: str) -> str:
    if len(v) >= 2 and v[0] == v[-1] and v[0] in ('"', "'"):
        return v[1:-1]

```

```

    return v

def parse(path: PathLike) -> Dict[str, str]:
    p = Path(path)
    out: Dict[str, str] = {}
    if not p.exists():
        return out
    for line in p.read_text(encoding="utf-8").splitlines():
        s = line.strip()
        if not s or s.startswith("#") or "=" not in s:
            continue
        if s.startswith("export "):
            s = s[len("export "):].strip()
        k, v = s.split("=", 1)
        out[k.strip()] = _strip_quotes(v.strip())
    return out

def load(path: PathLike, override: bool = False) -> None:
    """Populate os.environ from a .env file (existing vars win unless override)."""
    for k, v in parse(path).items():
        if override or k not in os.environ:
            os.environ[k] = v

def write_var(path: PathLike, key: str, value: str) -> None:
    """Insert or update KEY=value in place, preserving all other lines. chmod 600."""
    p = Path(path)
    lines = p.read_text(encoding="utf-8").splitlines() if p.exists() else []
    out, found = [], False
    for line in lines:
        s = line.strip()
        name = None
        if s and not s.startswith("#") and "=" in s:
            lhs = s.split("=", 1)[0].strip()
            name = lhs[len("export "):].strip() if lhs.startswith("export ") else lhs
        if name == key:
            out.append(f"{key}={value}")
            found = True
        else:
            out.append(line)
    if not found:
        out.append(f"{key}={value}")
    p.write_text("\n".join(out) + "\n", encoding="utf-8")
    try:
        os.chmod(p, 0o600) # owner read/write only - keys aren't world-readable
    except OSError:
        pass

def ensure_gitignore(env_path: PathLike) -> bool:
    """Make sure the .env filename is gitignored beside it. Returns True if added."""
    env_p = Path(env_path)
    gi = env_p.parent / ".gitignore"
    name = env_p.name
    existing = gi.read_text(encoding="utf-8").splitlines() if gi.exists() else []
    if any(line.strip() == name for line in existing):
        return False
    with gi.open("a", encoding="utf-8") as fh:
        if existing and existing[-1].strip() != "":
            fh.write("\n")
        fh.write(name + "\n")
    return True

```

triad/keychain.py

"""Optional OS keyring storage (macOS Keychain, Windows Credential Locker, etc.) via the `keyring` package. Every function degrades gracefully when `keyring` isn't installed, so importing this module is always safe.

```

    pip install keyring # to enable
"""
from __future__ import annotations

import os
from typing import Iterable, Optional

```

```

_SERVICE = "triad"

def available() -> bool:
    try:
        import keyring # noqa: F401
        return True
    except Exception:
        return False

def set_key(env: str, value: str) -> bool:
    try:
        import keyring
        keyring.set_password(_SERVICE, env, value)
        return True
    except Exception:
        return False

def get_key(env: str) -> Optional[str]:
    try:
        import keyring
        return keyring.get_password(_SERVICE, env)
    except Exception:
        return None

def load_missing(env_names: Iterable[str]) -> int:
    """Fill os.environ from the keyring for any names not already set."""
    n = 0
    for name in env_names:
        if not os.environ.get(name):
            v = get_key(name)
            if v:
                os.environ[name] = v
                n += 1
    return n

```

triad/setup_keys.py

"""Interactive API-key setup: masked prompt -> live validation -> safe save.

```

python -m triad setup          # core providers (OpenAI / Anthropic / Gemini)
python -m triad setup --all    # also free-cloud providers (NIM / Groq / OpenRouter)
python -m triad setup --no-validate
python -m triad setup --reconfigure

```

Validation hits each provider's `GET /models` endpoint, which confirms the key works without spending any tokens. A bad key is caught before it's saved.

```

"""
from __future__ import annotations

import os
import urllib.error
import urllib.request
from getpass import getpass
from pathlib import Path
from typing import Callable, Dict, List, Optional, Tuple

from . import dotenv, keychain

# Validation result: True = valid, False = rejected, None = couldn't verify (e.g. offline).
Validation = Tuple[Optional[bool], str]
_REJECT = {400, 401, 403}

def _check(url: str, headers: Dict[str, str], timeout: float = 10.0) -> Validation:
    req = urllib.request.Request(url, headers=headers, method="GET")
    try:
        with urllib.request.urlopen(req, timeout=timeout) as r:
            return True, f"valid ({r.status})"
    except urllib.error.HTTPError as e:
        if e.code in _REJECT:
            return False, f"key rejected ({e.code})"
        if e.code == 429:
            return True, "valid (rate-limited)"

```

```

        return None, f"unverified (HTTP {e.code})"
    except Exception as e: # DNS / timeout / offline
        return None, f"unverified ({type(e).__name__})"

def _v_openai(k: str) -> Validation:
    return _check("https://api.openai.com/v1/models", {"Authorization": f"Bearer {k}"})

def _v_anthropic(k: str) -> Validation:
    return _check("https://api.anthropic.com/v1/models",
                  {"x-api-key": k, "anthropic-version": "2023-06-01"})

def _v_gemini(k: str) -> Validation:
    return _check("https://generativelanguage.googleapis.com/v1beta/models",
                  {"x-goog-api-key": k})

def _v_nim(k: str) -> Validation:
    return _check("https://integrate.api.nvidia.com/v1/models", {"Authorization": f"Bearer {k}"})

def _v_groq(k: str) -> Validation:
    return _check("https://api.groq.com/openai/v1/models", {"Authorization": f"Bearer {k}"})

def _v_openrouter(k: str) -> Validation:
    return _check("https://openrouter.ai/api/v1/models", {"Authorization": f"Bearer {k}"})

class Provider:
    def __init__(self, name: str, env: str, signup: str, validate: Callable[[str], Validation]):
        self.name = name
        self.env = env
        self.signup = signup
        self.validate = validate

CORE: List[Provider] = [
    Provider("OpenAI (ChatGPT)", "OPENAI_API_KEY",
            "https://platform.openai.com/api-keys", _v_openai),
    Provider("Anthropic (Claude)", "ANTHROPIC_API_KEY",
            "https://console.anthropic.com/settings/keys", _v_anthropic),
    Provider("Google (Gemini)", "GEMINI_API_KEY",
            "https://aistudio.google.com/apikey", _v_gemini),
]

# Free-cloud providers (for routing free-claude-code or other gateways).
EXTRA: List[Provider] = [
    Provider("NVIDIA NIM", "NVIDIA_NIM_API_KEY",
            "https://build.nvidia.com/settings/api-keys", _v_nim),
    Provider("Groq", "GROQ_API_KEY", "https://console.groq.com/keys", _v_groq),
    Provider("OpenRouter", "OPENROUTER_API_KEY", "https://openrouter.ai/keys", _v_openrouter),
]

def mask(key: str) -> str:
    if not key:
        return "(not set)"
    if len(key) <= 8:
        return "*" * len(key)
    return f"{key[:3]}...{key[-4:]}"

def _prompt_key(prompt: str) -> str:
    try:
        return getpass(prompt).strip()
    except (EOFError, KeyboardInterrupt):
        raise
    except Exception:
        # Fallback for terminals where getpass can't hide input.
        return input(prompt).strip()

def run_setup(env_path, providers: List[Provider], validate: bool = True,
              reconfigure: bool = False) -> None:
    env_path = Path(env_path)
    existing = dotenv.parse(env_path)

```

```

use_keychain = False
if keychain.available():
    ans = input("Store keys in your OS keychain instead of a plaintext .env? [y/N]
").strip().lower()
    use_keychain = ans in ("y", "yes")

print()
saved = 0
for p in providers:
    current = existing.get(p.env) or os.environ.get(p.env, "") or (
        keychain.get_key(p.env) if use_keychain else "")

    if current:
        print(f"{p.name}: currently {mask(current)}")
        entry = _prompt_key(" paste a new key, or press Enter to keep: ") if not reconfigure \
            else _prompt_key(" paste key (Enter to skip): ")
    else:
        print(f"{p.name}: {mask('')} - get one at {p.signup}")
        entry = _prompt_key(" paste key (Enter to skip): ")

    if not entry:
        print(" kept.\n" if current and not reconfigure else " skipped.\n")
        continue

    if validate:
        ok, detail = p.validate(entry)
        symbol = {True: "[ok]", False: "[x]", None: "?"}[ok]
        print(f" {symbol} {detail}")
        if ok is False:
            if input(" save anyway? [y/N] ").strip().lower() not in ("y", "yes"):
                print(" not saved.\n")
                continue

    if use_keychain:
        keychain.set_key(p.env, entry)
    else:
        dotenv.write_var(env_path, p.env, entry)
        os.environ[p.env] = entry # live for the current process too
        saved += 1
        print(" saved.\n")

if not use_keychain and env_path.exists():
    if dotenv.ensure_gitignore(env_path):
        print(f"Added {env_path.name} to .gitignore")

where = "your OS keychain" if use_keychain else str(env_path)
print(f"Done - {saved} key(s) saved to {where}.")

```

triad/cli.py

"""Interactive REPL.

Plain text -> broadcast as a task in the current mode.
 /commands -> control modes, skills, agents, history.

A single asyncio event loop runs the whole session; blocking input() is offloaded
 to a thread so streaming stays responsive and provider clients stay bound to one loop.

```

"""
from __future__ import annotations

import argparse
import asyncio
import os
import sys
from pathlib import Path
from typing import Dict, List

from rich.console import Console

from . import dotenv, keychain, vault
from .agents import Agent, build_agents
from . import config
from .orchestrator import Orchestrator
from .setup_keys import CORE, EXTRA, run_setup
from .skills import Skill, load_skills

DEFAULT_ENV = Path(__file__).resolve().parent.parent / ".env"

```

```

HELP = """[bold]Commands[/bold]
  /mode parallel|relay|council|verify  switch how agents collaborate
  /oracle <cmd> | off                  verify-mode pass condition (e.g. /oracle "pytest -q")
  /protocol on|off                     compact handoffs in relay/council (saves tokens)
  /skill <name> [agent|all]            apply a skill (default target = its frontmatter)
  /skills                             list available skill files
  /clearskills                         strip all applied skills
  /agents                             list active agents + models
  /keys                               add or fix an API key (re-runs setup)
  /reset                              clear conversation history
  /remember                           write the session to the Obsidian vault now (needs --vault)
  /save [file.md]                     save transcript
  /help                               this help
  /quit                               exit

[bold]Modes[/bold]
  parallel  same task to all three at once, independent answers side by side
  relay     agents work in sequence, each building on the previous output
  council   all answer, then a chair agent synthesizes the best single answer
  verify    generate candidates, run each against an oracle in the sandbox, keep a passer
            (critique-revise if none); no oracle -> "unverified, selection only" (set with /oracle)
"""

def _apply_skill(parts: List[str], agents: List[Agent], skills: Dict[str, Skill], console: Console):
    if len(parts) < 2:
        console.print("usage: /skill <name> [agent|all]")
        return
    sk = skills.get(parts[1])
    if not sk:
        console.print(f"[red]no skill named '{parts[1]}'[/red] - try /skills")
        return
    targets = [parts[2]] if len(parts) > 2 else sk.agents
    applied = []
    for a in agents:
        if "all" in targets or a.name in targets:
            a.add_skill(sk.body)
            applied.append(a.label)
    console.print(f"applied [bold]{sk.name}[/bold] -> {' '.join(applied) or '(no agent matched)'}")

def _activate_new(agents: List[Agent], console: Console, roster: str = "paid") -> None:
    """Append providers whose keys just appeared. `agents` is the live list shared with
    the orchestrator, so append exactly once - appending to both aliases double-adds."""
    have = {a.name for a in agents}
    for a in build_agents(roster):
        if a.name not in have:
            agents.append(a)
            console.print(f"added [bold]{a.color}{a.label}[/]")

def _save(orch: Orchestrator, filename: str, console: Console):
    if not orch.transcript:
        console.print("[dim]nothing to save yet[/dim]")
        return
    body = "# triad transcript\n\n" + "\n".join(orch.transcript)
    with open(filename, "w", encoding="utf-8") as fh:
        fh.write(body)
    console.print(f"saved -> [bold]{filename}[/bold]")

def _command(line: str, orch: Orchestrator, agents, skills, console) -> bool:
    """Return True to quit."""
    parts = line.split()
    c = parts[0].lower()
    if c in ("/quit", "/exit", "/q"):
        return True
    if c == "/help":
        console.print(HELP)
    elif c == "/mode":
        if len(parts) > 1 and parts[1] in ("parallel", "relay", "council", "verify"):
            orch.mode = parts[1]
            console.print(f"mode -> [bold]{orch.mode}[/bold]")
        else:
            console.print("usage: /mode parallel|relay|council|verify")
    elif c == "/oracle":
        if len(parts) > 1 and parts[1].lower() in ("off", "none", "clear"):
            orch.oracle = None
            console.print("oracle cleared - verify is now [yellow]unverified (selection

```

```

only)[/yellow]")
    elif len(parts) > 1:
        from .oracle import CommandOracle
        cmd = line.split(None, 1)[1]
        orch.oracle = CommandOracle(cmd)
        console.print(f"oracle set: [bold]{cmd}[/bold] (use it in [bold]/mode verify[/bold])")
    else:
        cur = orch.oracle.describe() if orch.oracle else "none (unverified - selection only)"
        console.print(f"oracle: {cur}\nusage: /oracle <cmd> | /oracle off")
elif c == "/protocol":
    if len(parts) > 1 and parts[1] in ("on", "off"):
        orch.protocol = parts[1] == "on"
        state = "on" if orch.protocol else "off"
        console.print(f"protocol -> [bold]{state}[/bold] (affects relay/council)")
    else:
        console.print(f"protocol is [bold]{'on' if orch.protocol else 'off'}[/bold] - usage:
/protocol on|off")
elif c == "/agents":
    for a in agents:
        console.print(f" [bold]{a.color}]{a.label}[/] ({a.name}) model={a.model}")
elif c == "/skills":
    if not skills:
        console.print("[dim]no skill files found[/dim]")
    for s in skills.values():
        console.print(f" [bold]{s.name}[/bold] -> {' '.join(s.agents)}
[dim]{s.description}[/dim]")
elif c == "/skill":
    _apply_skill(parts, agents, skills, console)
elif c == "/clearskills":
    for a in agents:
        a.set_system("")
    console.print("cleared all applied skills")
elif c == "/reset":
    for a in agents:
        a.clear()
    console.print("conversation history cleared")
elif c == "/save":
    _save(orch, parts[1] if len(parts) > 1 else "triad_transcript.md", console)
else:
    console.print(f"[red]unknown command {c}[/red] - try /help")
return False

def _input(prompt: str) -> str:
    try:
        return input(prompt)
    except (EOFError, KeyboardInterrupt):
        return "/quit"

async def repl(args) -> None:
    console = Console()
    roster = getattr(args, "roster", "paid")
    providers = {"paid": CORE, "free": EXTRA, "all": CORE + EXTRA}[roster]

    # Make keys available without the user having to `source` anything.
    dotenv.load(DEFAULT_ENV)
    keychain.load_missing([p.env for p in providers])

    agents = build_agents(roster)
    if not agents:
        console.print(f"[yellow]No API keys found for the {roster} roster yet.[/yellow]")
        ans = (await asyncio.to_thread(_input, "Run setup now? [Y/n] ")).strip().lower()
        if ans in ("", "y", "yes"):
            await asyncio.to_thread(run_setup, DEFAULT_ENV, providers, True, False)
            agents = build_agents(roster)
        if not agents:
            console.print("Still no keys. Run [bold]python -m triad setup[/bold] when ready.")
            return

    skills = load_skills(args.skills_dir)
    orch = Orchestrator(agents, skills, console, mode=args.mode, chair=args.chair)
    if args.oracle:
        from .oracle import CommandOracle
        orch.oracle = CommandOracle(args.oracle)

    names = ", ".join(f"[bold]{a.color}]{a.label}[/]" for a in agents)
    console.rule("[bold]triad[/bold]")
    console.print(f"agents: {names} roster: [bold]{roster}[/bold]")

```

```

console.print(f"mode: [bold]{orch.mode}[/bold]   skills loaded: {len(skills)}   (/help for
commands)")
if roster in ("free", "all"):
    console.print(
        "[yellow]free roster:[/yellow] [dim]aggregating free models is only trustworthy with a "
        "verifier (run the code/tests). Without one they make correlated errors and then agree - "
        "
        "use for verifiable tasks; don't judge free council on its own.[/dim]")
if args.seed_file:
    resume = vault.read_resume(args.seed_file)
    if resume:
        for a in agents:
            a.add_skill(f"## Resume context (from
{vault.resume_path(args.seed_file)})\n{resume}")
        console.print(f"resume: loaded {len(resume)} chars from
[bold]{vault.resume_path(args.seed_file)}[/bold]")
    else:
        console.print(f"[yellow]resume: nothing read from {args.seed_file}[/yellow]")
if args.vault:
    vault.open_vault(args.vault, agents, skills)
    console.print(f"vault: live notes -> [bold]{args.vault}[/bold] "
        "(open the folder as an Obsidian vault; Cmd/Ctrl+G for the graph)")
console.print()

while True:
    line = (await asyncio.to_thread(_input, f"triad[{orch.mode}]> ")).strip()
    if not line:
        continue
    if line.lower() in ("/keys", "/key"):
        await asyncio.to_thread(run_setup, DEFAULT_ENV, providers, True, False)
        for a in agents: # apply fixes to keys already in use
            val = os.environ.get(config.resolve(a.name)["key_env"], "")
            if val and val != a.api_key:
                a.api_key, a._client = val, None
                console.print(f"updated key for [bold]{a.color}{a.label}[/]")
            _activate_new(agents, console, roster) # add providers whose keys just appeared (once)
        console.print()
        continue
    if line.lower() in ("/remember", "/vault"):
        if args.vault:
            n = vault.remember(args.vault, agents, orch)
            console.print(f"remembered -> [bold]{args.vault}[/bold] ({n} agent note(s)
updated)")
        else:
            console.print("no vault - start triad with [bold]--vault <dir>[/bold]")
            continue
    if line.startswith("/"):
        if _command(line, orch, agents, skills, console):
            break
        continue
    try:
        await orch.dispatch(line)
        if args.vault:
            vault.remember(args.vault, agents, orch) # live note appears within a second
    except KeyboardInterrupt:
        console.print("\n[yellow]interrupted - turn aborted (Ctrl-C again at the prompt to
quit)[/yellow]")
    except Exception as e:
        console.print(f"[red]dispatch error:[/red] {type(e).__name__}: {e}")
    console.print()

if args.vault:
    vault.remember(args.vault, agents, orch) # final flush on exit
console.print("bye ■")

def _run_setup_cli(argv: List[str]) -> None:
    sp = argparse.ArgumentParser(prog="triad setup",
                                description="Configure API keys (masked input, live validation).")
    sp.add_argument("--all", action="store_true", help="also configure free-cloud providers")
    sp.add_argument("--no-validate", dest="validate", action="store_false",
                    help="skip the live key check")
    sp.add_argument("--reconfigure", action="store_true", help="re-prompt every key, even ones
already set")
    a = sp.parse_args(argv)
    providers = CORE + (EXTRA if a.all else [])
    run_setup(DEFAULT_ENV, providers, validate=a.validate, reconfigure=a.reconfigure)

def main() -> None:

```

```

argv = sys.argv[1:]
if argv and argv[0] == "setup":
    _run_setup_cli(argv[1:])
    return

p = argparse.ArgumentParser(prog="triad",
                             description="Talk to ChatGPT, Claude, and Gemini at once.")
p.add_argument("--mode", default="parallel", choices=["parallel", "relay", "council", "verify"])
p.add_argument("--oracle", default=None, metavar="CMD",
               help='verify-mode pass condition, run against each candidate in the sandbox, '
                    'e.g. --oracle "pytest -q"')
p.add_argument("--roster", default="paid", choices=["paid", "free", "all"],
               help="paid=OpenAI/Anthropic/Gemini, free=Groq/OpenRouter/NIM, all=both")
p.add_argument("--free", action="store_const", const="free", dest="roster",
               help="shorthand for --roster free")
p.add_argument("--skills-dir", default=os.path.join(os.path.dirname(os.path.dirname(__file__)),
            "skills"))
p.add_argument("--chair", default=None, help="agent name to synthesize in council mode (default: first)")
p.add_argument("--vault", default=None, metavar="DIR",
               help="write a live Obsidian vault of the session to DIR (notes per agent + topics + index)")
p.add_argument("--seed-file", dest="seed_file", default=None, metavar="DIR|FILE",
               help="load a vault's index.md (or any markdown file) back as resume context")
args = p.parse_args()
asyncio.run(repl(args))

if __name__ == "__main__":
    main()

```

triad/ __main__.py

```

# Load .env BEFORE importing cli (which imports config). config reads TRIAD_* / *_API_KEY
# at import time, so the .env values must already be in os.environ or those overrides - model
# IDs, TRIAD_CLAUDE_BASE_URL, etc. - are silently ignored. Real env vars still win (no override).
from pathlib import Path

from . import dotenv

dotenv.load(Path(__file__).resolve().parent.parent / ".env")

from .cli import main # noqa: E402 (must follow the .env load above)

if __name__ == "__main__":
    main()

```

triad/ __init__.py

```

"""triad - talk to ChatGPT, Claude, and Gemini at once from your terminal."""
__version__ = "0.1.0"

```

requirements.txt

```

rich>=13.7
openai>=1.30
anthropic>=0.40
google-genai>=0.3

```

.env.example

```

# Easiest: don't edit this by hand. Run: python -m triad setup
# (masked prompts, live key validation, writes .env with 600 perms + gitignores it)
#
# This file just documents the variable names. Copy to .env if you prefer manual setup.

# --- The three agents ---
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=...

# --- Optional free-cloud providers (python -m triad setup --all) ---
# NVIDIA_NIM_API_KEY=nvapi-...
# GROQ_API_KEY=gsk-...

```

```
# OPENROUTER_API_KEY=sk-or-...

# Model IDs change often - set whatever you actually want to run.
TRIAD_OPENAI_MODEL=gpt-4.1
TRIAD_CLAUDE_MODEL=claude-sonnet-4-6
TRIAD_GEMINI_MODEL=gemini-2.5-pro

# Optional
TRIAD_MAX_TOKENS=4096
```

skills/planner.md

```
---
name: planner
description: Decomposes a fuzzy task into a concrete, ordered plan
agents: [all]
---
You are a planning specialist. Before solving anything, break the task into a
short ordered plan:
1. Restate the goal in one sentence.
2. List the concrete steps needed, in order, each a single line.
3. Call out unknowns, assumptions, and the riskiest step.
Keep it tight. No filler. If the task is trivial, say so and skip the ceremony.
```

skills/architect.md

```
---
name: architect
description: Designs system structure and interfaces before any code
agents: [claude]
---
You are a software architect. Focus on structure, not implementation details.
Produce: the component breakdown, the data/control flow between them, the key
interfaces (function/class signatures), and the main trade-offs you're making.
Prefer simple, composable designs. Name the one decision most likely to be wrong
and what would change your mind.
```

skills/implementer.md

```
---
name: implementer
description: Writes clean, working, runnable code
agents: [chatgpt]
---
You are an implementation specialist. Write complete, runnable code - no
"...rest of the code here" placeholders. Match the existing style and stack.
Include only the comments that earn their place. After the code, give a 2-line
note on how to run it and anything you deliberately left out of scope.
```

skills/code-reviewer.md

```
---
name: code-reviewer
description: Hunts for bugs, edge cases, and security issues
agents: [gemini]
---
You are a rigorous code reviewer. Do NOT rewrite the whole thing. Instead list
concrete findings, each as: [severity] file/area - problem - suggested fix.
Severities: blocker / major / minor / nit. Prioritize correctness bugs, race
conditions, unhandled errors, and security issues over style. If it's solid,
say so and list only what you'd genuinely change.
```

skills/red-team.md

```
---
name: red-team
description: Adversarial pressure-testing - find how this fails
agents: [all]
---
You are a red-teamer. Assume the proposed answer/design/plan is flawed and find
how. Attack the assumptions, the edge cases, the failure modes, the things that
break under load or bad input or an adversary. Be specific and concrete - "this
fails when X" beats "consider robustness". End with the single most likely way
```

this goes wrong in practice.

skills/synthesizer.md

```
---
name: synthesizer
description: Merges multiple answers into one strong final answer (good council chair)
agents: [claude]
---
You are a synthesizer. Given several independent answers, produce one superior
answer. Combine the strongest parts, drop the weak ones, resolve contradictions
explicitly (state which you picked and why), and correct any errors you spot.
Don't average - judge. If the inputs genuinely disagree on something important,
surface that disagreement rather than papering over it.
```

skills/concise.md

```
---
name: concise
description: Style overlay - strips padding, forces directness
agents: [all]
---
Be direct and concise. No preamble, no "great question", no restating the prompt
back. Lead with the answer. Use prose, not bullet soup, unless structure
genuinely helps. Cut every sentence that doesn't carry information.
```